

CHUYÊN ĐỀ MÔN TIN HỌC

THUẬT TOÁN TÌM KIẾM THEO CHIỀU RỘNG VÀ ỨNG DỤNG

(Breadth-First Search – BFS)

Huỳnh Tấn Thông

Trường THPT chuyên Nguyễn Quang Diêu, tỉnh Đồng Tháp

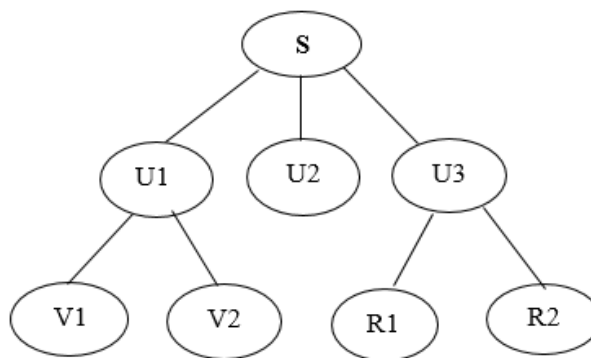
Email: huynhtanhtongtinhoc@gmail.com



I- THUẬT TOÁN TÌM KIẾM THEO CHIỀU RỘNG:

Một bài toán quan trọng trong lý thuyết đồ thị là bài toán duyệt tất cả các đỉnh có thể đến được từ một đỉnh xuất phát nào đó. Vấn đề này đưa về một bài toán liệt kê mà yêu cầu của nó là không được bỏ sót hay lặp lại bất kỳ đỉnh nào. Chính vì vậy mà ta phải xây dựng những thuật toán cho phép duyệt một cách hệ thống các đỉnh, những thuật toán như vậy gọi là những thuật toán tìm kiếm trên đồ thị (graph traversal). Ta quan tâm đến hai thuật toán cơ bản nhất: thuật toán tìm kiếm theo chiều sâu và thuật toán tìm kiếm theo chiều rộng, trong khuôn khổ chuyên đề này tác giả chỉ trình bày thuật tìm kiếm theo chiều rộng và ứng dụng của nó và được minh họa bằng ngôn ngữ C/C++

Ví dụ: Cho $G = \{V, E\}$; như hình minh họa.



1.1. Ý tưởng:

Tư tưởng của thuật toán Tìm kiếm theo chiều rộng (Breadth-First Search – BFS) là “lập lịch” duyệt các đỉnh. Việc thăm một đỉnh sẽ lên lịch duyệt các đỉnh nối từ nó sao cho thứ tự duyệt là ưu tiên chiều rộng (đỉnh nào gần đỉnh xuất phát S hơn sẽ được duyệt trước). Đầu tiên ta thăm đỉnh S. Việc thăm đỉnh S phát sinh thứ tự thăm những đỉnh U1, U2, U3 là những đỉnh kề với đỉnh S. Tiếp theo thăm đỉnh U1, khi thăm đỉnh U1 lại phát sinh yêu cầu thăm những đỉnh V1, V2 là các đỉnh kề với đỉnh U1. Nhưng rõ ràng các đỉnh V này “xa” đỉnh S hơn những đỉnh U nên chúng chỉ được thăm khi tất cả những đỉnh U kề với đỉnh S đã được thăm. Thứ tự duyệt các đỉnh là: S; U1; U2; U3; V1; V2; R1; R2.

Thuật toán Tìm kiếm theo chiều rộng sử dụng một danh sách để chứa những đỉnh đang “chờ” thăm. Tại mỗi bước, ta thăm một đỉnh đầu danh sách, loại nó ra khỏi danh sách và cho những đỉnh chưa “xếp hàng” kề với nó xếp hàng thêm vào cuối danh sách. Thuật toán sẽ kết thúc khi danh sách rỗng. Vì nguyên tắc vào trước ra trước ta sử dụng hàng đợi (queue): Trong C/C++ đã xây dựng thư viện queue, vector và các hàm/ thủ tục để thao tác đến chúng. Do đó để thuận lợi và đem lại tính hiệu quả tác giả dùng ngôn ngữ lập trình C/C++ để giải các bài toán.

1.2. Thuật toán:

Thuật toán BFS	Code minh họa
<pre>void bfs(int x){ Khai báo một hàng đợi Đánh dấu đỉnh x đã thăm Đẩy đỉnh x vào hàng đợi Lặp cho đến khi hàng đợi rỗng{ + Lấy đỉnh u từ hàng đợi + Xóa đỉnh u ra khỏi hàng đợi + Xét tất cả đỉnh v kề đỉnh u + Nếu đỉnh v chưa thăm thì { Thăm đỉnh v// lưu đỉnh v Đẩy đỉnh v vào hàng đợi } } }</pre>	<pre>void bfs(int x){ queue<int> q; q.push(x); ch[x]=1; while (!q.empty()){ int u=q.front(); q.pop(); for(int i=0; int v=a[u][i]; i++) if (ch[v]==0) ch[v]=1; //tr[v]=u; q.push(v); } }</pre>



1.3. Cấu trúc dữ liệu và độ phức tạp của thuật toán:

- Cấu trúc dữ liệu: Thuật toán BFS có thể dùng ma trận kề hoặc vector (mảng động) để biểu diễn đồ thị, bên cạnh đó thuật toán còn dùng mảng đánh dấu, để đánh dấu những đỉnh đã thăm, để lưu những đỉnh trung gian chờ duyệt thuật toán dùng một hàng đợi.

- Độ phức tạp của thuật toán BFS phụ thuộc vào biểu diễn đồ thị: nếu biểu diễn đồ thị bằng ma trận kề thì: $O(V^2)$, nếu biểu diễn bằng vector thì: $O(|V|+|E|)$.

1.4. Tính chất của BFS và ứng dụng:

- Thuật toán BFS luôn trả về đường đi qua ít cạnh nhất trong số tất cả các đường đi từ **S** tới **T**. Nếu ta sắp xếp các danh sách kề của mỗi đỉnh theo thứ tự tăng dần và nếu có nhiều đường đi từ **S** tới **T** đều qua ít cạnh nhất thì thuật toán BFS sẽ trả về đường đi có thứ tự từ điển nhỏ nhất trong số những đường đi đó.

- Thuật toán Tìm kiếm theo chiều rộng được dùng để giải nhiều bài toán trong lý thuyết đồ thị, chẳng hạn như:

- + Liệt kê tất cả các đỉnh trong một thành phần liên thông/ đếm số thành phần liên thông của đồ thị
- + Tìm đường đi ngắn nhất giữa hai đỉnh **S** và **T** (với chiều dài đường đi tính bằng số lượng cạnh)
- + Kiểm tra xem một đồ thị có phải là đồ thị hai phía
- + Thuật toán Ford–Fulkerson để tìm luồng cực đại trong mạng
- + Thuật toán BFS kết hợp với các thuật toán khác để giải các bài toán phức hợp.

II. ỨNG DỤNG CỦA THUẬT TOÁN BFS:

2.1. Thành phần liên thông của đồ thị:

Bài 1. BÁNH KẸO (Olympic 30/4/2018 Tin 10)

Công ty bánh kẹo ABC chuẩn bị xây dựng hệ thống đại lý để giao bánh kẹo đến tất cả địa điểm trong thành phố. Hàng ngày, từ mỗi đại lý các nhân viên dùng **K** loại xe với trọng lượng lần lượt là $P_1, P_2, \dots, P_K$ chuyên chở bánh kẹo đến các địa điểm còn lại. Thành phố có

N địa điểm (đánh số từ 1 đến N) và M con đường hai chiều nối trực tiếp giữa các địa điểm, trên mỗi con đường có ghi một số nguyên dương cho biết trọng lượng tối đa của xe được phép di chuyển trên con đường này. Giữa 2 địa điểm bất kì có thể có nhiều con đường nối trực tiếp.

Để có thể giao hàng đến N địa điểm, công ty tìm cách chọn một số địa điểm để đặt đại lý. Chi phí mỗi cách chọn phụ thuộc vào số lượng địa điểm được chọn làm đại lý, số địa điểm càng ít thì chi phí càng thấp.

Yêu cầu: Hãy cho biết cách chọn có chi phí thấp nhất sẽ có số lượng địa điểm đặt đại lý là bao nhiêu.

Dữ liệu vào: Cho trong file văn bản **BANHKEO.INP**.

+ Dòng 1 gồm ba số nguyên N ($1 \leq N \leq 1000$), M ($1 \leq M \leq 100000$), K ($1 \leq K \leq 20$).

+ Dòng 2 gồm K số nguyên dương $P_1, P_2, \dots, P_K$ ($1 \leq P_i \leq 500$)

+ M dòng tiếp theo, dòng thứ i gồm ba số nguyên dương A_i, B_i, T_i ($1 \leq T_i \leq 500$), cho biết con đường nối giữa địa điểm A_i đến B_i chịu được trọng lượng tối đa là T_i . Các số ghi trên cùng một dòng cách nhau bởi ít nhất một kí tự trắng.

Kết quả: Ghi ra file văn bản **BANHKEO.OUT** gồm một dòng với số nguyên duy nhất cho biết số lượng địa điểm ít nhất.

Ví dụ:

BANHKEO.INP	BANHKEO.OUT	GIẢI THÍCH
5 6 3 5 3 4 1 2 2 1 3 1 2 3 3 3 4 2 4 5 2 4 5 4	3	Công ty có ba loại xe với trọng lượng 5, 3, 4 Bằng cách chọn ba địa điểm 1, 3, 5 để đặt đại lý. Công ty có thể giao bánh kẹo đến tất cả các địa điểm.

a- Phân tích bài toán: Xây dựng đơn đồ thị vô hướng với trọng số như sau:

- Các địa điểm là một đỉnh đồ thị, Trọng tải các con đường là độ dài các cạnh của đồ thị. Do đề bài cho là một đa đồ thị do đó khi xây dựng đồ thị ta cần chọn cạnh lớn nhất giữa hai đỉnh (u,v) để biểu diễn đơn đồ thị.

- Khi nhân viên đi giao hàng ta chọn xe có tải trọng nhỏ nhất để đi được nhiều địa điểm nhất. ta có những vùng liên thông, mỗi vùng liên thông chọn một địa điểm làm đại lý ta được chi phí thấp nhất theo đề yêu cầu.

- Kết quả bài toán là số lượng vùng liên thông của đồ thị.

- Bài toán cho $n, m \leq 10^4$ do đó ta chỉ cần biểu diễn đồ thị bằng ma trận kề.

b- Code tham khảo:

```
#include <bits/stdc++.h>
using namespace std;
const int N = 10000+1;
int a[N][N], n, m, k, res;
int pmin=1e9;
bool ch[N];
```



```

void doctep(){
    cin>>n>>m>>k;
    for(int i=1; i<=k ; i++){
        int p;
        cin>>p;
        pmin=min(pmin, p);
    }
    for(int i=1; i<=m; i++){
        int x, y, z;
        cin>>x>>y>>z;
        if(a[x][y]<z)
            a[x][y]=a[y][x]=z;
    }
}
void bfs(int x){
    queue <int> q;
    q.push(x);
    ch[x]=1;
    while(!q.empty()){
        int u=q.front();
        q.pop();
        for(int v=1; v<=n; v++){
            if(a[u][v]>=pmin && ch[v]==0){
                q.push(v);
                ch[v]=1;
            }
        }
    }
}
main(){
    freopen("BANHKEO.INP", "r",stdin);
    freopen("BANHKEO.OUT","w", stdout);
    doctep();
    for(int i=1; i<=n; i++){
        if(ch[i]==0){
            res=res+1;
            bfs(i);
        }
    }
    cout<<res;
}

```



2.2. Đường đi qua ít cạnh nhất kết hợp quy hoạch động:

Bài 2. TÌNH YÊU VÀ CÂY CÀU DỪA

An Tiêm đang sinh sống trên một quần đảo **Nga Sơn** gồm **N** đảo. Các đảo khá gần nhau nên chẳng cần thuyền bè, An Tiêm chỉ cần đón đại cây dừa nào đó và bắc ngang là có thể đi được từ đảo này sang đảo khác mất **K** thời gian. Nếu chưa bắc được cầu thì An Tiêm di chuyển qua các đảo bằng cách bơi bướm hay bơi ngựa đồng thời cũng là để tập thể dục

luôn, nhưng Bạn gái của An Tiêm thì không biết bơi. Hiện tại An Tiêm mới bắc được **M** cây cầu dừa.

Vì lo lắng An Tiêm sẽ buồn chán khi ở đảo hoang, bạn gái An Tiêm từ trong đất liền dự định sang chơi với anh ấy với mục tiêu là nhanh nhất.

Nhưng mà "Môi sơn, má đào, chân guốc cao gót làm sao em qua cầu dừa???". Lo lắng sợ bạn gái sẽ rơi xuống biển và bị cá mập nuốt mất. An Tiêm biết các bạn đội Tin rất giỏi lập trình nên đã nhờ các bạn tìm đường đi nhanh nhất từ đảo **S** đến đảo **T** mà An Tiêm đang ở.

Input: Tập văn bản Love.inp

- Dòng thứ nhất: chứa 4 số nguyên **N, M, S, T** và **K**
- Tiếp theo M dòng: mỗi dòng gồm hai số nguyên **a, b**. Trong đó **a** và **b** là số hiệu hai đảo được nối bởi duy nhất một cây cầu dừa.

Output: Ghi vào tập văn bản Love.out

Chỉ có một dòng ghi hai số nguyên là thời gian ít nhất đi từ đảo **S** đến **F** và số lượng đường đi với thời gian ít nhất đã tìm được.

Ví dụ:

LOVE.INP	LOVE.OUT
4 4 1 4 5	10 2
1 2	
2 4	
1 3	
3 4	

Ràng buộc: $1 \leq S, T \leq N \leq 10000$; $1 \leq M \leq 1000000$; $K \leq 10^9$

a- Phân tích bài toán:

- Xây dựng đồ thị, mỗi đảo là một đỉnh, mỗi cây cầu dừa là một cạnh của đồ thị.
- Bài toán đặt ra là tìm đường đi từ đỉnh **S** đến đỉnh **T** qua ít cạnh nhất và bao nhiêu con đường đi như thế:

+ Thuật toán BFS cho ta đường đi từ đỉnh **S** đến đỉnh **T** qua ít cạnh nhất, trong thuật toán dùng thêm mảng **d[]** khi duyệt đến đỉnh **v** thì **d[v]=d[u]+1** (với u là đỉnh thăm trước v).

+ Đếm số đường đi ngắn nhất ta dùng quy hoạch động: Gọi mảng **sl[]** là mảng qui hoạch động có ý nghĩa: đếm số lượng đường đi từ đỉnh **S** đến đỉnh **T**, theo nguyên lý toán học **sl[v]=sl[u]+sl[v]** (với u là đỉnh thăm trước v).

- Giới hạn của bài: $N \leq 10000$; $M \leq 1000000$ ta cần dùng vector để biểu diễn đồ thị.

b- Code tham khảo:

```
#include <bits/stdc++.h>
using namespace std;
const int N=1e4+1;
vector <int> a[N];
int n, m, s, t, k;
bool ch[N];
long long d[N];
long long sl[N];
```



```

void doctep(){
    cin>>n>>m>>s>>t>>k;
    for(int i=1;i<=m; i++){
        int x, y;
        cin>>x>>y;
        a[x].push_back(y);
        a[y].push_back(x);
    }
    for (int i=0; i<=n; i++) a[i].push_back(0);
}
void bfs(int s){
    queue <int> q;
    q.push(s);
    ch[s]=1;
    sl[s]=1;
    while(!q.empty()){
        int u=q.front();
        if(u==t) return;
        q.pop();
        for(int i=0; int v=a[u][i]; i++){
            if (ch[v]==0){
                q.push(v);
                ch[v]=1;
                d[v]=d[u]+1;
                sl[v]=sl[u]+sl[u];
            }
            else
                if(u!=v && d[u]+1==d[v]) sl[v]=sl[u]+sl[v];
        }
    }
}
main(){
    ios::sync_with_stdio(false);
    freopen("LOVE.INP", "r", stdin);
    freopen("LOVE.OUT", "w", stdout);
    doctep();
    bfs(s);
    cout<<d[t]*k<<" "<<sl[t];
}

```



2.3. Đường đi ngắn nhất kết hợp với tính chất của cây:

Bài 3. BENEFACT – The Benefactor

Ngày xưa ngày xưa, tại một thành phố, người dân quyết định lựa chọn ra một vị thị trưởng. Tại thời điểm đó, có một ý tưởng rất mới và chiến dịch bầu cử hoàn toàn chưa biết. Nhưng dĩ nhiên một vài người dân muốn trở thành thị trưởng. Và nó không mất thời gian để nhận ra rằng việc đưa ra những lời hứa tốt sẽ không trở nên có hiệu quả. Một ứng cử viên được chọn nếu như những món quà giá trị của ông ta đem lại lợi ích cho những người dân trong thành phố.

Một ngày trước ngày bầu cử, một ứng cử viên đã hứa với người dân trong thành phố. Nếu như chiến thắng trong cuộc bầu cử thì anh ta sẽ đảm bảo rằng: tại một trong những con đường của thành phố sẽ được lắp đèn. Và anh ta sẽ trả mọi khoản phí. Người dân cuối cùng đã lựa chọn anh ta. Và một ngày sau ngày bầu cử, người dân đã trình bày quyết định của họ về tuyến đường nào sẽ được chọn. Dĩ nhiên họ không chỉ chọn đoạn đường dài nhất trong thành phố mà họ còn thay đổi tên của chúng để được một đoạn đường dài nhất có thể.

Bạn có thể tìm ra độ dài của đoạn phố này không? Để chi tiết hơn, bạn được đưa cho danh sách của các đoạn đường. Cho biết giữa hai điểm chỉ có một đoạn đường duy nhất. Nhiệm vụ của bạn là tìm ra khoảng cách lớn nhất giữa hai điểm trong thành phố.

Input: BENEFACT.INP

Dòng đầu tiên của đầu vào bao gồm 1 số T là số lượng test cases. Dòng đầu tiên của mỗi test case bao gồm số lượng các điểm n với $2 \leq n \leq 50000$. Mỗi đường được đưa ra trên một dòng bởi 2 điểm a, b ($1 \leq a, b \leq n$) và độ dài của nó l ($0 \leq l < 20000$).

Output: BENEFACT.OUT Với mỗi testcase, in ra kết quả cần tìm.

Ví dụ:

BENEFACT.INP	BENEFACT.OUT
1	12
6	
1 2 3	
2 3 4	
2 6 2	
6 4 6	
6 5 5	

a- Phân tích:

+ Ở đây, số lượng điểm tối đa là 50000, do đó ta bắt buộc phải dùng vector. Để đã cho là một cây có n nút thì có đúng n-1 cạnh và giữa hai nút chỉ có duy nhất một đường đi.

+ Thuật toán để giải quyết bài toán là: ta sẽ sử dụng thuật toán tìm kiếm theo chiều rộng – BFS 2 lần. Lần thứ nhất, BFS tại 1 điểm bất kỳ, giả sử là điểm 1. Sau lần này ta sẽ tìm ra điểm ở xa điểm 1 nhất, gọi là **dmax1**. Lần thứ hai, ta sẽ BFS từ điểm **dmax1**. Tiếp tục ta lại tìm được một điểm nằm xa điểm **dmax1** nhất, gọi là **dmax2**. Khoảng cách giữa **dmax1** và **dmax2** là kết quả cần tìm.

b- Code tham khảo:

```
#include <bits/stdc++.h>
using namespace std;
const int N=5*1e4;
typedef pair <int, int> ii;
int m,n, dmax, dd;
vector <ii> a[N];
bool ch[N];
int d[N];
void nhap(){
    int x, y, z;
    cin>>n;
    for(int i=1; i<=n-1; i++){
```



```

        cin>>x>>y>>z;
        a[x].push_back(ii(y, z));
        a[y].push_back(ii(x, z));
    }
    for(int i=0; i<=n; i++) a[i].push_back(ii(0,0));
}
void bfs(int x){
    dmax=0;
    memset(d, 0, sizeof(d));
    memset(ch, 0, sizeof(ch));
    queue<int> q;
    int u, v;
    q.push(x);
    ch[x]=1;
    while (!q.empty()){
        u=q.front();
        q.pop();
        for(int i=0; v=a[u][i].first; i++) // duyệt các đỉnh v kề u
            if (ch[v]==0){
                q.push(v);
                ch[v]=1;
                d[v]=d[u]+a[u][i].second;
                if (d[v]>dmax){
                    dmax=d[v];
                    dd=v;
                }
            }
    }
}
}
main(){
    freopen("BENEFAC.T.INP", "r", stdin);
    freopen("BENEFAC.T.OUT", "w", stdout);
    int t;
    cin>>t;
    for (int k=1; k<=t; k++){
        memset(a, 0, sizeof(a));
        nhap();
        bfs(1);
        bfs(dd);
        cout<<d[dd]<<endl;
    }
}

```



2.4. Đường đi ngắn nhất kết hợp tính chất của hình học:

Bài 4. TRÒ CHƠI LÒ CÒ (Đề Olympic Bắc Bộ)

Lò Cò (hay cò cò) là một trò chơi dân gian, được cho là đã có từ thời La Mã cổ đại, rất thông dụng và có ảnh minh họa trên các giáo đường. Trò chơi này rèn luyện người mới chơi tập trung giữ thăng bằng, nâng cao sự khéo léo và tính toán. Cụ thể, trên sân được vẽ N hình chữ nhật, các hình chữ nhật được đánh số từ 1 đến N . Người chơi cần di chuyển từ hình 1 đến hình N với luật di chuyển như sau: nếu người chơi đang ở hình thứ i thì có thể di chuyển tới hình thứ j nếu hình i và hình chữ j có phần diện tích giao nhau khác 0.

Yêu cầu: Tìm cách di chuyển qua từ hình 1 đến hình N mà qua ít hình chữ nhật nhất.

Dữ liệu: Vào từ file văn bản GOS.INP:

- Dòng 1: chứa số N ($N \leq 1000$) là số hình chữ nhật;
- N dòng sau, mỗi dòng chứa 4 số nguyên t_x, t_y, p_x, p_y ($|t_x|, |t_y|, |p_x|, |p_y| \leq 10^6$), trong đó (t_x, t_y) là tọa độ tâm của hình chữ nhật và (p_x, p_y) là tọa độ của một đỉnh của hình chữ nhật.

Kết quả: Ghi ra file văn bản GOS.OUT một số là số bước nhảy ít nhất để di chuyển từ hình 1 đến hình thứ N , nếu không di chuyển được thì ghi -1.

Ví dụ:

GOS.INP	GOS.OUT
3 0 0 1 1 1 0 2 1 4 0 7 2	2
3 0 0 1 1 2 0 3 1 4 0 5 1	-1

a- Phân tích bài toán:

Coi mỗi hình chữ nhật là một đỉnh của đồ thị, đỉnh i (hình chữ i) đến được đỉnh j (hình chữ nhật j) nếu hai hình chữ nhật có phần diện tích giao nhau khác 0.

Dùng thuật toán BFS trên đồ thị để tìm đường đi ngắn nhất từ đỉnh 1 đến đỉnh n .

b- Code tham khảo:

```
#include <bits/stdc++.h>
using namespace std;
const int N=1e4+4;
struct hcn{
    int tx, ty, px,py;
};
hcn a[N];
int c[N][N], d[N], n;
queue <int> q;
bool ch[N];
void doctep(){
    cin>>n;
    for (int i=1; i<=n; i++)
```



```

        cin>>a[i].tx>>a[i].ty>>a[i].px>>a[i].py;
    }
    bool kt(int i, int j){
        int dd, d1, d2;
        dd=abs(a[i].tx- a[j].tx); //khoan cach tam 2 hcn;
        d1=abs(a[i].tx - a[i].px); // duong cheo hcn 1
        d2=abs(a[j].tx-a[j].px); // duong cheo hcn2
        if (dd>=d1+d2) return 0;

        dd=abs(a[i].ty- a[j].ty); //khoan cach tam 2 hcn;
        d1=abs(a[i].ty - a[i].py); // duong cheo hcn 1
        d2=abs(a[j].ty-a[j].py); // duong cheo hcn2
        if (dd>=d1+d2) return 0;
        return 1;
    }
    void khoitao(){
        for( int i=1; i<n; i++)
            for (int j=i+1; j<=n; j++)
                if( kt(i, j)){
                    c[i][j]=c[j][i]=1;
                }
    }
    void bfs(int x){
        q.push(x);
        ch[x]=1; d[x]=0;
        while(!q.empty()){
            int u=q.front();
            q.pop();
            for(int v=1; v<=n; v++)
                if(c[u][v]==1 && ch[v]==0){
                    ch[v]=1;
                    q.push(v);
                    d[v]=d[u]+1;
                }
        }
    }

    }
    main(){
        freopen("GOS.INP","r", stdin);
        freopen("GOS.OUT","w", stdout);
        doctep();
        khoitao();
        bfs(1);
        if (ch[n]==0) cout<<-1;
        else cout<<d[n];
    }

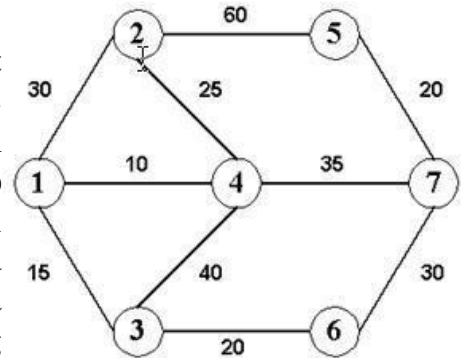
```



2.5. Tìm kiếm nhị phân kết hợp BFS:

Bài 5: HƯỚNG DẪN VIÊN DU LỊCH

Ông HTT. là một hướng dẫn viên du lịch. Công việc của ông ta là hướng dẫn một vài “tua” du lịch từ thành phố này đến thành phố khác. Trên các thành phố này, có một vài con đường hai chiều được nối giữa chúng. Mỗi cặp thành phố có đường kết nối đều có dịch vụ xe buýt chỉ chạy giữa hai thành phố này và chạy theo đường nối trực tiếp giữa chúng. Mỗi dịch vụ xe buýt đều có một giới hạn lớn nhất lượng khách mà xe buýt có thể trở được. Ông HTT có một tấm bản đồ chỉ các thành phố và những con đường nối giữa chúng. Ngoài ra, ông ta cũng có thông tin về mỗi dịch vụ xe buýt giữa các thành phố. Ông hiểu rằng ông không thể đưa tất cả các khách du lịch đến thành phố thăm quan trong cùng một chuyến đi. Lấy ví dụ: Về bản đồ gồm 7 thành phố, mỗi cạnh được nối giữa các thành phố biểu thị những con đường và các số viết trên mỗi cạnh cho biết cho biết giới hạn lớn nhất hành khách của dịch vụ xe buýt chạy trên tuyến đường đó.



Bây giờ, nếu ông HTT muốn đưa 99 khách du lịch từ thành phố 1 đến thành phố 7. Ông ta sẽ phải yêu cầu ít nhất là 5 chuyến đi, và lộ trình ông ta nên đi là 1 – 2 – 4 – 7.

Nhưng, Ông HTT. nhận thấy là thật khó để tìm ra tất cả lộ trình tốt nhất để sao cho ông ta có thể đưa tất cả khách du lịch đến thành phố thăm quan với số chuyến đi là nhỏ nhất. Do vậy mà ông ta cần sự trợ giúp của các bạn.

Dữ liệu: Vào từ file **Tourist.inp**

- Dòng đầu tiên chứa hai số nguyên N ($2 \leq N \leq 1000$) và R mô tả lần lượt số thành phố và số đường đi giữa các thành phố ($1 < R \leq 10000$).

- R dòng tiếp theo, mỗi dòng chứa 3 số nguyên: $C1, C2, P$. $C1, C2$ mô tả lộ trình đường đi từ thành phố $C1$ đến thành phố $C2$ và P là giới hạn có thể phục vụ của dịch vụ xe buýt giữa hai thành phố ($1 < P < 1000$).

- Các thành phố được đánh dấu bằng một số nguyên từ 1 đến N . Dòng thứ $(R+1)$ chứa ba số nguyên S, T, L mô tả lần lượt thành phố khởi hành, thành phố cần đến và số khách du lịch được phục vụ. ($1 < L < 10000$).

Kết quả: Đưa ra file **Tourist.out**

Ghi ra số lộ trình nhỏ nhất cần phải đi qua các thành phố thỏa mãn yêu cầu đề bài.

Ví dụ:

TOURIST.INP	TOURIST.OUT
7 10	5
1 2 30	
1 3 15	
1 4 10	
2 4 25	
2 5 60	
3 4 40	
3 6 20	
4 7 35	
5 7 20	

6 7 30	
1 7 99	

a- Phân tích:

- Với con đường (u,v) gọi $A[u][v]$ là số người tối đa có thể đi trên con đường đó trong một lần là $A[u][v]-1$ sẽ là số khách tối đa có thể đi trên con đường đó trong một lần. $A[u,v] = 0$ tương đương với giữa u và v không có con đường nào. Vậy số khách tối đa đi một lần nằm trong đoạn $[1..l]$ mà $l \leq 10000$ ta cần dùng thuật toán tìm kiếm nhị phân tìm giá trị x nằm trong đoạn $[1..l]$ ta dùng BFS kiểm tra có duyệt được từ đỉnh s đến đỉnh t hay không, nếu hàm BFS trả về giá trị True ta chặt lên ngược lại ta chặt xuống.

- Giá trị x tìm được là số người nhiều nhất trong một tua, kết quả số chuyến là l/x và nốt còn dư ta cần thêm một chuyến nữa.

b- Code tham khảo:

```
#include <bits/stdc++.h>
using namespace std;
const int N=1e4;
bool ch[N];
int s, t, l, n, m, res, a[N][N];
void doctep(){
    cin >> n >> m;
    for(int i=1; i<=m; i++){
        int x, y, z;
        cin >> x >> y >> z;
        a[x][y]=a[y][x]=z-1;
    }
    cin >> s >> t >> l;
}
bool bfs(int g){
    memset(ch,0, sizeof(ch));
    queue <int> q;
    ch[s]=1;
    q.push(s);
    while( !q.empty()){
        int u=q.front();
        q.pop();
        for(int v=1; v<=n; v++){
            if(a[u][v]>=g && ch[v]==0){
                q.push(v);
                ch[v]=1;
            }
        }
    }
    if (ch[t]==1) return 1;
    else return 0;
}
main(){
    freopen("TOURIST.INP","r",stdin);
    freopen("TOURIST.OUT","w",stdout);
```



```
doctep();
int dau=1, cuoi=l, g;
while (dau<=cuoi){
    g=(dau+cuoi)/2;
    if (bfs(g)==1){
        res=g;
        dau=g+1;
    }
    else cuoi=g-1;
}
if (l%res==0) cout<<(l/res);
else cout<<(l/res)+1;
}
```

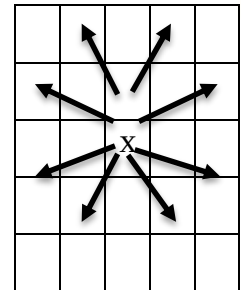


2.6. Loang trong bảng:

Bài 6: QUÂN MÃ (Olympic 30/4/2016 – Tin 10)

Trong luật cờ vua, mỗi nước đi của quân mã được quy định như sau: quân mã đang ở tại vị trí X như hình bên dưới có thể đi đến một trong các ô mà mũi tên chỉ đến (theo đường chéo của hình chữ nhật 2x3).

Yêu cầu: Cho trước bàn cờ kích thước $n \times m$. Hãy đếm số nước đi ít nhất để quân mã di chuyển từ ô có tọa độ $(x1, y1)$ đến ô có tọa độ $(x2, y2)$. Trong trường hợp không đến được thì xuất ra giá trị -1.



Dữ liệu vào: từ tệp văn bản QUANMA.INP gồm

- Dòng 1 ghi 2 số nguyên dương n, m ($2 \leq n, m \leq 1000$).
- Dòng 2 ghi 2 số nguyên $x1, y1$ ($1 \leq x1 \leq n; 1 \leq y1 \leq m$).
- Dòng 3 ghi 2 số nguyên $x2, y2$ ($1 \leq x2 \leq n; 1 \leq y2 \leq m$).

Các số ghi trên cùng một dòng cách nhau ít nhất một kí tự trắng.

Kết quả: Ghi ra tệp văn bản QUANMA.OUT một số nguyên duy nhất cho biết số nước đi ít nhất để quân mã di chuyển từ vị trí $(x1, y1)$ đến vị trí $(x2, y2)$. Nếu không đến được thì ghi giá trị là -1.

Ví dụ:

QUANMA.INP	QUANMA.OUT
4 6 1 1 2 4	2

a- Phân tích bài toán:

Xây dựng đồ thị gồm mỗi đỉnh là một ô của bàn cờ. Giữa hai đỉnh (x, y) và (x', y') có cạnh nối nếu trong một nước đi, quân mã có thể đi từ ô (x, y) đến (x', y') (tối đa 8 ô kề với x, y) hoặc ngược lại.

Sau đó ta duyệt BFS từ đỉnh $(x1, y1)$ đến $(x2, y2)$ để tìm số nước đi ít nhất. Nếu không đến được

b- Code tham khảo:

```

#include <bits/stdc++.h>
using namespace std;
const int N=1e3+3;
struct ma{
    int x, y;
};
int a[N][N]; // khai bao ban co
// mang 8 nuoc di cua quan ma theo ox, oy
int hx[8]={ -2,-2, 2, 2, -1, -1, 1, 1}; // gan gia tri truoc cho mang
int hy[8]={ -1, 1, -1, 1, -2, 2, -2, 2};
queue <ma> q; // khai bao hang doi kieu quan ma co 2 diem
int x1, y1, x2, y2, n, m;
void doctep(){
    cin>>n>>m;
    cin>>x1>>y1;
    cin>>x2>>y2;
}
void bfs(ma p){
    memset(a,-1, sizeof(a)); // khoi mang danh dau va dem = -1
    a[x1][y1]=0; // vi tri xuat phat dem = 0
    q.push(p); // day diem p vao queu
    while (!q.empty()){ // trong khi queue khac rong lam
        ma u=q.front(); // lay u ra khoi que
        q.pop();
        for(int v=0; v<8; v++){ // xet cac dinh v ke u // luu y chay tu 0
            ma t;
            t.x=u.x+hx[v];
            t.y=u.y+hy[v];
            // kiem tra co vuot ban co ko va chua di den
            if (t.x>0 && t.x<=n && t.y>0 && t.y<=m && a[t.x][t.y]==-1){
                a[t.x][t.y]=a[u.x][u.y]+1; //danh dau da toi va tang buoc di
                if (t.x==x2 && t.y==y2) break; // di den dinh dich thoat
                q.push(t);
            }
        }
    }
}
main(){
    freopen("quanma.inp","r", stdin);
    freopen("quanma.out","w", stdout);
    doctep();
    ma p;
    p.x=x1;
    p.y=y1;
    bfs(p);
    cout<<a[x2][y2];
}

```



2.7. Loang quản lí hướng:

Bài 7. ĐIỆP VIÊN (Đề thi chọn đội dự thi quốc gia 2018 của tỉnh Đồng Tháp)

Bản đồ địa bàn hoạt động của một điệp viên là một khu vực mà ở đó chỉ có các đường phố hai chiều Tây↔Đông và Bắc↔Nam tạo thành một lưới ô vuông đơn vị. Với mục đích bảo mật, thay vì tên đường phố, điệp viên đánh số các phố Tây↔Đông từ 0 đến m từ Bắc xuống Nam và các phố Bắc↔Nam từ 0 đến n từ Tây sang Đông. Giao của đường Tây↔Đông thứ i và đường Bắc↔Nam thứ j là nút giao thông (i, j) . Ở một số nút giao thông có đặt trạm kiểm soát. Điệp viên đang đứng nút giao thông (x_1, y_1) và cần tới điểm hẹn ở nút giao thông (x_2, y_2) . Để tránh bị theo dõi, đường đi phải không qua các trạm kiểm soát và cứ tới chỗ rẽ thì nhất thiết phải đổi hướng đi, thậm chí có thể đi ngược trở lại. Việc đổi hướng chỉ được thực hiện ở các nút giao thông.

Đường đi được mã hóa dưới dạng một dãy ký tự, trong đó ký tự thứ k thuộc tập ký tự $\{E, W, S, N\}$ cho biết tại bước đi chuyển thứ k , điệp viên sẽ phải đi một đơn vị độ dài theo hướng Đông (E), Tây (W), Nam (S) hay Bắc (N).

Yêu cầu: Hãy xác định đường đi ngắn nhất tới điểm hẹn. Nếu có nhiều đường đi ngắn nhất, hãy cho biết đường đi có thứ tự từ điển nhỏ nhất trong số đó. Biết rằng dữ liệu được cho để luôn tồn tại đường đi thỏa mãn yêu cầu đặt ra.

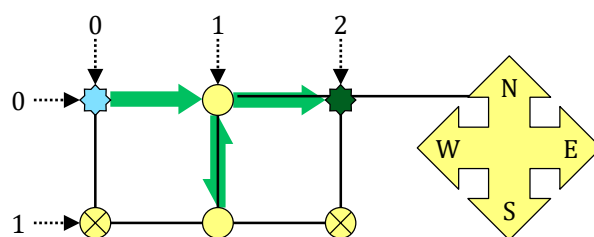
Dữ liệu: Vào từ file văn bản SPY.INP

- Dòng thứ nhất chứa 6 số nguyên: m, n, x_1, y_1, x_2, y_2 ($0 \leq m, n \leq 1000$; $0 \leq x_1, x_2 \leq m$; $0 \leq y_1, y_2 \leq n$)
- Các dòng sau, mỗi dòng chứa hai số tương ứng với tọa độ một trạm kiểm soát (chỉ số đường Tây↔Đông và chỉ số đường Bắc↔Nam)

Kết quả: Ghi ra file văn bản SPY.OUT đường đi tìm được.

Ví dụ:

SPY.INP	SPY.OUT
1 2 0 0 2	ESNE
1 0	
1 2	



Code tham khảo:

```
#include <bits/stdc++.h>
#define taskname "SPY"
using namespace std;
const int maxMN = 1001;
const int maxD = 4;
int dx[maxD] = {0, -1, 1, 0};
int dy[maxD] = {1, 0, 0, -1};
int m, n, xs, ys, xt, yt;
bool a[maxMN][maxMN][maxD];
```



```

int trace[maxMN][maxMN][maxD];
struct TNode{
    int x, y, d;
};
TNode EndNode;
inline int ReadInt(){
    int c;
    for (c = getchar(); c < '0' || c > '9'; c = getchar())
        if (c == EOF) return -1;
    int res = c - '0';
    for (c = getchar(); c >= '0' && c <= '9'; c = getchar())
        res = res * 10 + c - '0';
    return res;
}
void Enter(){
    m = ReadInt(); n = ReadInt();
    xs = ReadInt(); ys = ReadInt();
    xt = ReadInt(); yt = ReadInt();
    fill_n(&a[0][0][0], sizeof(a) / sizeof(a[0][0][0]), true);
    while (true) {
        int x, y;
        x = ReadInt();
        if (x < 0) break;
        y = ReadInt();
        for (int d = 0; d < maxD; ++d) a[x][y][d] = false;
    }
}

bool Valid(int x, int y, int d){
    return x >= 0 && x <= m && y >= 0 && y <= n && a[x][y][d];
}

TNode Solve(){
    if (xs == xt && ys == yt){
        return {xt, yt, maxD};
    }
    queue<TNode> q;
    q.push({xs, ys, maxD});
    while (true) {
        TNode u = q.front(); q.pop();
        for (int d = 0; d < maxD; ++d)
            if (d != u.d) {
                TNode v = {u.x + dx[d], u.y + dy[d], d};
                if (Valid(v.x, v.y, v.d)){
                    a[v.x][v.y][v.d] = false;
                    trace[v.x][v.y][v.d] = u.d;
                    if (v.x == xt && v.y == yt)
                        return v;
                }
            }
    }
}

```



```

        q.push(v);
    }
}
}
}
void DoTrace(){
    vector<char> res;
    TNode u = EndNode;
    while (u.x != xs || u.y != ys) {
        switch (u.d){
            case 0: res.push_back('E'); break;
            case 1: res.push_back('N'); break;
            case 2: res.push_back('S'); break;
            case 3: res.push_back('W'); break;
        }
        int prevd = trace[u.x][u.y][u.d];
        u.x -= dx[u.d];
        u.y -= dy[u.d];
        u.d = prevd;
    }
    for (auto it = res.rbegin(); it != res.rend(); ++it)
        putchar(*it);
}
main(){
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    freopen(taskname".inp", "r", stdin);
    freopen(taskname".out", "w", stdout);
    Enter();
    EndNode = Solve();
    DoTrace();
}

```



III. KẾT LUẬN

Thuật toán tìm kiếm theo chiều rộng (BFS) được ứng dụng rất nhiều trong các bài toán đồ thị. Sự cải tiến và kết hợp của thuật toán với các thuật toán khác có thể đem lại nhiều điều thú vị, giải quyết được nhiều lớp bài toán khác nhau, phần nào cho thấy được tầm quan trọng của thuật toán BFS.

Với khả năng và năng lực còn hạn chế, trong chuyên đề chưa trình bày hết các ứng dụng của thuật toán BFS như: kiểm tra đồ thị có phải là đồ thị hai phía, kết hợp với thuật toán Ford–Fulkerson, Cheney, Cuthill–McKee, ... Các bài giải có thể chưa sáng, chưa tối ưu và bài tập trong chuyên đề này được tổng hợp từ nhiều nguồn tài liệu với hy vọng thông qua chuyên đề tác giả đã truyền tải đến đồng nghiệp một phần nào đó cách sử dụng và kết hợp của thuật toán Tìm kiếm theo chiều rộng để giải các bài toán về đồ thị.

Thành phố Cao Lãnh, ngày 08/6/2018

Người viết

Huỳnh Tấn Thông

Tài liệu tham khảo:

- [1] Tài liệu Chuyên Tin quyển 1, 2, 3 – Hồ Sĩ Đàm (chủ biên)
- [2] Cấu trúc Dữ liệu và Giải thuật – Lê Minh Hoàng (ĐHSP Hà Nội)
- [3] Tài liệu Hội thảo Khoa học Khu vực Duyên Hải, Bắc Bộ năm 2013 (Nhiều tác giả)

